

Inbuilt Functions

Common Inbuilt Functions

print("Hello World")	# 1
len("Python")	# 2
type(10)	# 3
int("25")	# 4
float(5)	# 5
str(100)	# 6
abs(-20)	# 7
pow(2, 3)	# 8
round(3.456, 2)	# 9
max(10, 20, 30)	# 10
min(10, 20, 30)	# 11
sum([1, 2, 3, 4])	# 12
sorted([4, 1, 3, 2])	# 13
range(5)	# 14
list(range(5))	# 15
tuple([1, 2, 3])	# 16
set([1, 1, 2, 3])	# 17
dict(name="Athar", age=25)	# 18
enumerate(["a", "b", "c"])	# 19
zip([1, 2], [3, 4])	# 20

User-Defined Functions (20 Examples)

Functions created using def keyword.

Basic Functions

```
def greet():
```

```
    print("Hello Python")
```

```
greet()                # 21
```

```
def add(a, b):
```

```
    return a + b
```

```
print(add(5, 3))        # 22
```

```
def square(n):
```

```
    return n * n
```

```
print(square(4))        # 23
```

```
def even_odd(n):
```

```
    return "Even" if n % 2 == 0 else "Odd"
```

```
print(even_odd(7))      # 24
```

```
def find_max(a, b, c):
```

```
    return max(a, b, c)
```

```
print(find_max(10, 20, 15)) # 25
```

Functions with Default Arguments

```
def power(base, exp=2):
```

```
    return base ** exp
```

```
print(power(3))          # 26
```

Functions with Multiple Returns

```
def math_ops(a, b):
```

```
    return a+b, a-b, a*b
print(math_ops(10, 5))          # 27
```

Functions Using Loops

```
def factorial(n):
    f = 1
    for i in range(1, n+1):
        f *= i
    return f
print(factorial(5))             # 28
```

```
def count_vowels(word):
    count = 0
    for ch in word:
        if ch.lower() in "aeiou":
            count += 1
    return count
print(count_vowels("Python"))  # 29
```

List Processing Functions

```
def sum_list(lst):
    return sum(lst)
print(sum_list([1,2,3]))       # 30
```

```
def reverse_list(lst):
    return lst[::-1]
print(reverse_list([1,2,3]))   # 31
```

```
def unique_items(lst):
    return list(set(lst))
print(unique_items([1,1,2,3])) # 32
```

```
def average(lst):  
    return sum(lst)/len(lst)  
print(average([10,20,30]))    # 33  
  
def find_length(s):  
    return len(s)  
print(find_length("Athar"))    # 34
```

Function with Input

```
def area_circle(r):  
    return 3.14 * r * r  
print(area_circle(5))          # 35  
  
def check_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return "Not Prime"  
    return "Prime"  
print(check_prime(7))          # 36  
  
def string_upper(s):  
    return s.upper()  
print(string_upper("python"))  # 37  
  
def multiply_list(lst):  
    result = 1  
    for i in lst:  
        result *= i  
    return result  
print(multiply_list([1,2,3,4])) # 38  
  
def greet_user(name):  
    return f"Welcome {name}"
```

```
print(greet_user("Athar"))      # 39
```

Lambda (Anonymous) Functions (10 Examples)

Lambda functions are one-line functions.

Syntax

lambda arguments: expression

Examples

```
add = lambda a, b: a + b
```

```
print(add(2, 3))              # 40
```

```
square = lambda x: x * x
```

```
print(square(5))              # 41
```

```
even = lambda n: n % 2 == 0
```

```
print(even(8))                # 42
```

```
max_num = lambda a, b: a if a > b else b
```

```
print(max_num(10, 20))        # 43
```

```
length = lambda s: len(s)
```

```
print(length("Python"))       # 44
```

Lambda with map()

```
nums = [1,2,3,4]
```

```
squares = list(map(lambda x: x*x, nums))
```

```
print(squares)                # 45
```

Lambda with filter()

```
nums = [1,2,3,4,5,6]
```

```
evens = list(filter(lambda x: x%2==0, nums))
```

```
print(evens) # 46
```

Lambda with sorted()

```
pairs = [(1,3), (2,2), (4,1)]
```

```
sorted_pairs = sorted(pairs, key=lambda x: x[1])
```

```
print(sorted_pairs) # 47
```

```
cube = lambda x: x**3
```

```
print(cube(3)) # 48
```

```
discount = lambda price: price * 0.9
```

```
print(discount(1000)) # 49
```

```
check_length = lambda s: "Long" if len(s) > 5 else "Short"
```

```
print(check_length("Python")) # 50
```

```
a
```

INBUILT FUNCTIONS (51–65)

```
all([True, True, False]) # 51
```

```
any([False, False, True]) # 52
```

```
chr(65) # 53
```

```
ord('A') # 54
```

```
divmod(10, 3) # 55
```

```
id(100) # 56
```

```
isinstance(10, int) # 57
```

```
issubclass(bool, int) # 58
```

```
bin(10) # 59
```

```
oct(10) # 60
```

```
hex(10) # 61
```

```
help(len) # 62
```

```
hash("python")           # 63
```

```
repr("hello")            # 64
```

```
callable(print)          # 65
```

USER-DEFINED FUNCTIONS (66–90)

Mathematical Functions

```
def add_three(a, b, c):
```

```
    return a + b + c
```

```
print(add_three(1, 2, 3))    # 66
```

```
def is_positive(n):
```

```
    return n > 0
```

```
print(is_positive(5))        # 67
```

```
def simple_interest(p, r, t):
```

```
    return (p * r * t) / 100
```

```
print(simple_interest(1000, 5, 2))    # 68
```

```
def area_rectangle(l, w):
```

```
    return l * w
```

```
print(area_rectangle(5, 4))    # 69
```

```
def celsius_to_fahrenheit(c):
```

```
    return (c * 9/5) + 32
```

```
print(celsius_to_fahrenheit(25))    # 70
```

String Functions

```
def reverse_string(s):
```

```
    return s[::-1]
```

```
print(reverse_string("Python"))    # 71
```

```
def count_words(sentence):
```

```
    return len(sentence.split())

print(count_words("I love Python"))    # 72

def is_palindrome(s):

    return s == s[::-1]

print(is_palindrome("madam"))          # 73

def remove_spaces(s):

    return s.replace(" ", "")

print(remove_spaces("hello world"))    # 74

def first_last_char(s):

    return s[0], s[-1]

print(first_last_char("Python"))       # 75
```

List Functions

```
def find_largest(lst):

    return max(lst)

print(find_largest([10, 20, 5]))       # 76

def find_smallest(lst):

    return min(lst)

print(find_smallest([10, 20, 5]))      # 77

def count_even(lst):

    return len([x for x in lst if x % 2 == 0])

print(count_even([1,2,3,4,6]))        # 78

def square_list(lst):

    return [x*x for x in lst]

print(square_list([1,2,3]))            # 79

def merge_lists(a, b):

    return a + b

print(merge_lists([1,2], [3,4]))      # 80
```

Logic & Conditions

```
def grade(marks):
```

```
    if marks >= 90:
```

```
        return "A"
```

```
    elif marks >= 75:
```

```
        return "B"
```

```
    else:
```

```
        return "C"
```

```
print(grade(82))                # 81
```

```
def login(user, pwd):
```

```
    return user == "admin" and pwd == "1234"
```

```
print(login("admin", "1234"))    # 82
```

```
def leap_year(year):
```

```
    return year % 4 == 0
```

```
print(leap_year(2024))          # 83
```

```
def table(n):
```

```
    for i in range(1, 6):
```

```
        print(n, "x", i, "=", n*i)
```

```
table(5)                        # 84
```

```
def sum_n(n):
```

```
    return n * (n + 1) // 2
```

```
print(sum_n(10))                # 85
```

Dictionary Functions

```
def create_dict(keys, values):
```

```
    return dict(zip(keys, values))
```

```
print(create_dict(["a","b"], [1,2])) # 86
```

```
def dict_keys(d):
    return list(d.keys())

print(dict_keys({"a":1,"b":2}))    # 87

def dict_values(d):
    return list(d.values())

print(dict_values({"a":1,"b":2}))    # 88

def check_key(d, key):
    return key in d

print(check_key({"a":1}, "a"))    # 89

def dict_length(d):
    return len(d)

print(dict_length({"a":1,"b":2}))    # 90
```

LAMBDA FUNCTIONS (91–100)

```
double = lambda x: x * 2

print(double(5))    # 91

add_tax = lambda price: price + price*0.18

print(add_tax(1000))    # 92

greater = lambda a, b, c: max(a, b, c)

print(greater(5, 10, 3))    # 93

starts_with_a = lambda s: s.startswith("A")

print(starts_with_a("Athar"))    # 94

odd_numbers = list(filter(lambda x: x%2!=0, [1,2,3,4,5]))

print(odd_numbers)    # 95

upper_words = list(map(lambda s: s.upper(), ["python","java"]))

print(upper_words)    # 96

lengths = list(map(lambda s: len(s), ["hi","hello"]))

print(lengths)    # 97
```

```
salary_hike = lambda sal: sal * 1.1
print(salary_hike(50000))          # 98
check_age = lambda age: "Adult" if age >= 18 else "Minor"
print(check_age(20))               # 99
sort_by_length = sorted(["apple", "hi", "banana"], key=lambda x: len(x))
print(sort_by_length)              # 100
```

Summary

Type	Description
Inbuilt	Already available in Python
User-defined	Created using def
Lambda	One-line anonymous functions